

Canvas API 2D

Apunte y guía de cátedra: Versión 1.0

Introducción

La API Canvas 2D es una interfaz de programación de JavaScript que permite generar, renderizar y manipular gráficos bidimensionales de forma dinámica directamente en el navegador. Los objetos de tipo *HTMLCanvasElement*, funcionan como un lienzo de dibujo donde el desarrollador utiliza un contexto gráfico de renderizado (2d) para dibujar mediante código formas geométricas tales como rectas, curvas, arcos, texto, etc; como así también representar texturas, composición y efectos visuales. En complementación con objetos temporizadores, posibilita la construcción de animaciones de cualquier grado de complejidad. A diferencia de tecnologías basadas en dibujo vectorial como SVG, Canvas opera bajo un modelo de gráficos de mapa de bits (gráficos rasterizados).

Aplicaciones

La API Canvas 2D se utiliza principalmente en el desarrollo de experiencias interactivas y dinámicas en tiempo real, como videojuegos en dos dimensiones, simuladores y animaciones complejas que requieren altas tasas de fotogramas por segundo. Asimismo, es una herramienta fundamental para la visualización y representación gráfica de datos, permitiendo generar desde diagramas e infografías estadísticas hasta tableros con métricas en vivo. Por otro lado, destaca en el procesamiento y manipulación de contenido visual, donde se emplea tanto para la edición y aplicación de filtros sobre imágenes y video como para la creación de herramientas de dibujo o la exportación de gráficos generados directamente desde el navegador.

Contextualización

Si bien la API Canvas 2D puede operarse desde cualquier nivel de abstracción, nuestro objetivo en este espacio de cursada es que por un lado aprenda a identificar concretamente las diferentes funcionalidades básicas y elementales de manipulación de gráficos bidimensionales que proporciona esta API, pero por otro lado, también lograr una conceptualización e integración con las decisiones de arquitectura y diseño de software que fuimos abordando durante el recorrido de la materia. Por lo tanto, el aprendizaje de esta unidad no debe considerarse como un conocimiento aislado, sino situado e integrado dentro del tópico central de la asignatura del diseño y desarrollo de aplicaciones basadas en WebComponents con arquitectura MVC.

Arquitectura MVC

La arquitectura MVC como se ha desarrollado con anterioridad, es en esencia una arquitectura que se basa en la separación de responsabilidades en tres aspectos fundamentales. El modelo es el responsable de proveer la abstracción de la aplicación desde la perspectiva de la información (estado) y su lógica de operación. La vista es la responsable de proporcionar la representación gráfica del modelo y el espacio de interacción. Y por último, el controlador es el responsable de gestionar las interacciones del usuario en la vista y en realizar las interacciones con el modelo subyacente según corresponda. La idea de recapitular estas definiciones es que usted pueda ir infiriendo la ubicación en donde debe residir el objeto *HTMLCanvasElement* dentro de esta arquitectura y en dónde debe gestionarse el estado de forma adecuada. Muchas veces, la literatura o fuentes de internet contextualizan la arquitectura MVC de forma muy sesgada, dando a entender que se refiere a aplicaciones de tipo web principalmente del ecosistema de gestión y/o aplicaciones de escritorio. Pocas veces advertirá el uso de la arquitectura MVC en el contexto de juegos y/o alto rendimiento gráfico y generalmente encontrará que se utilizan arquitecturas del tipo ECS (Entity-Component-System). Más allá de la terminología y el marketing, no debe perder de vista los fundamentos y reconocer que siempre la arquitectura MVC propone esencialmente una separación de responsabilidades y no una implementación concreta de cómo llevarlo a cabo.

Aspectos a tener en cuenta

La presente guía de ejercitación tiene varios propósitos y se encuentra separada por tópicos progresivos. En primer lugar, la primera tanda de ejercicios propone ofrecer una familiarización hacia los nuevos conocimientos y progresivamente estos le irán exigiendo integración con saberes previos. Más allá de que esté familiarizado por sus propios saberes personales, o no, no se recomienda que saltee ejercicios, ni que extienda de más lo que se le pide en cada uno de ellos. Por otro lado, se espera que no utilice inteligencia artificial generativa para su resolución por varias razones. En primer lugar, no le servirá para construir su propio conocimiento ni para fijar conceptos básicos. Por otro lado, se evaluarán estos contenidos bajo modalidad de examen escrito sin computadora y mediante un ejercicio práctico integrador.

Requisitos previos a la resolución de la guía de ejercitación

Para iniciar la resolución de la guía de trabajo, deberá disponer de una plantilla en blanco de un WebComponent vacío expresado bajo la arquitectura MVC. A lo largo de la primera clase, se construirá colectivamente la encapsulación del elemento `HTMLCanvasElement` y su utilización esencial a modo de ejemplificación.

Modalidad de entrega

La guía de ejercitación deberá ser entregada mediante el repositorio de trabajo de la asignatura, dentro de la carpeta denominada *unidad-4*. Dentro de la misma deberá colocar un archivo *MVCCanvas.js* que contendrá el WebComponent de la plantilla mencionada anteriormente y será el que deberá importar en cada una de las soluciones de los ejercicios solicitados como única dependencia. Cada ejercicio puntual se resolverá en un único archivo de nombre: *"ejercicio-x.js"* cuya *x* corresponde al número correspondiente. Ejemplo: *ejercicio-1.js*, *ejercicio-2.js*, etc. Por otro lado, deberá mantener un archivo *index.html* que despliegue el ejercicio completo marcado como **integración**.

Evaluación

El docente considerará aprobada la unidad temática, de acuerdo a los siguientes requisitos:

- a) Entrega completa de todos los ejercicios solicitados.
- b) Adecuación de las normas de entrega.
- c) Aprobación del examen escrito y/o recuperatorio (sin computadora) con nota mayor e igual a 7 (siete).
- d) Aprobación del ejercicio final de integración.

Capítulo 1: Sistema de coordenadas y primitivas de dibujo

Las primitivas de dibujo constituyen los bloques de construcción fundamentales en Canvas 2D, permitiendo transformar instrucciones abstractas de código en elementos visuales sobre el lienzo. La API basa toda su geometría en dos tipos principales de primitivas: *rectángulos* y *caminos* (paths), que actúan como trazados invisibles compuestos por segmentos de líneas rectas, arcos y curvas (Bezier y cuadráticas). A partir de la combinación de estas figuras básicas, y mediante el uso de operaciones de dibujado como el rellenado (*fill*) para definir áreas sólidas o el contorneado (*stroke*) para trazar bordes, se puede estructurar y componer cualquier objeto gráfico, por más complejo que sea, píxel a píxel.

Para comenzar a dibujar, lo primero y esencial es construir el objeto canvas y familiarizarse con el sistema de posicionamiento gráfico:

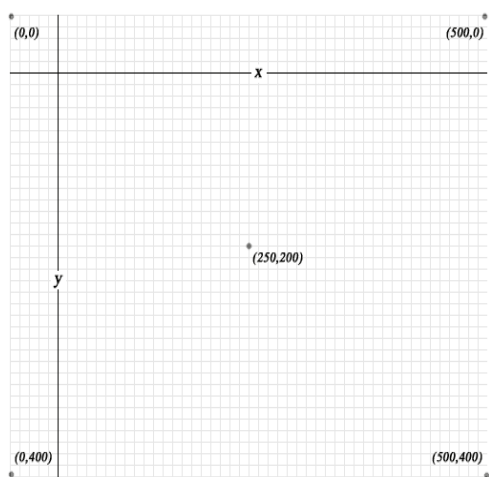
```
// 1. Crear el elemento HTMLCanvasElement
const canvas = document.createElement('canvas');

// 2. Definir las dimensiones del lienzo (ancho y alto en píxeles)
canvas.width = 800;
canvas.height = 600;

// 3. Obtener el contexto de renderizado 2D
const ctx = canvas.getContext('2d');

// 4. Insertar el canvas en el árbol del DOM (ej. dentro del <body>)
document.body.appendChild(canvas);
```

Observará un simple y ligero rectángulo blanco sin más. Sin embargo, deberá interpretarlo de la siguiente forma:



El vértice superior izquierdo se corresponde con el origen del centro de coordenadas, equivalente a la posición (0,0).

El eje horizontal (x) adquiere valores positivos en sentido hacia la derecha.

El eje vertical (y) adquiere valores positivos en sentido hacia abajo.

Cualquier coordenada que contenga componentes absolutas negativas, implica que el dibujado se inicia por fuera de la región visible. A veces se utiliza esa estrategia para “invisibilizar” dibujos, aunque no siempre es recomendable.

El uso de coordenadas cartesianas y absolutas, puede ser suficiente para muchos casos, aunque suelen promover rigidez y problemas de composición cuando se reutilizan abstracciones de dibujo. En esos casos, se emplean sistemas de coordenadas relativos. Esto se verá más adelante cuando alcance el capítulo de Transformaciones Geométricas.

El objeto canvas es una abstracción que representa “una hoja de dibujo y una mano con un lápiz”. Las operaciones primitivas básicas son las siguientes:

Categoría	Método	Descripción
Composición de trazados	beginPath()	Inicia un nuevo camino o trazado, limpiando cualquier sub-camino previo en memoria.
	closePath()	Añade una línea recta desde la posición actual hasta el punto de inicio del camino actual para cerrarlo.
	fill()	Rellena el interior del camino actualmente acumulado en memoria.
	stroke()	Pinta el contorno del camino actualmente acumulado en memoria.
Movimiento	moveTo(x, y)	Mueve el lápiz a la coordenada (x, y) especificada sin trazar ninguna línea.
Recta	lineTo(x, y)	Agrega una línea recta al camino desde la posición actual hasta la coordenada (x, y).
Rectángulo	rect(x, y, ancho, alto)	Agrega el trazado de un rectángulo al camino actual sin pintarlo directamente.
	fillRect(x, y, ancho, alto)	Dibuja y rellena inmediatamente un rectángulo en el lienzo.
	strokeRect(x, y, ancho, alto)	Dibuja inmediatamente el contorno de un rectángulo en el lienzo.
Arco	arc(x, y, radio, anguloInicio, anguloFin)	Agrega el trazado de un arco o círculo al camino actual indicando centro, radio y ángulos en radianes.
	arcTo(x1, y1, x2, y2, radio)	Agrega un arco tangente entre la posición actual y dos puntos de control.
Texto	fillText(texto, x, y)	Escribe y rellena inmediatamente una cadena de texto en la posición indicada.
	strokeText(texto, x, y)	Dibuja inmediatamente el contorno de un texto en la posición indicada.

El método beginPath() lo que hacen es comenzar a guardar temporalmente la trayectoria de trazos que usted irá desarrollando posteriormente por medio de los métodos: line, arc, rectangle y moveTo. Esto conforma un “camino de trazos invisibles” que puede cerrarse aplicando el método closePath() si aplica. Sin embargo, el trazado sólo se materializa visualmente con la aplicación del método stroke() para aplicar contorno o fill() para llenar el área resultante. A estos métodos se los conoce como métodos de renderizado. Aún así, nada le impide ir aplicando renderizados de contorno y relleno a medida que va generando trazos intermedios.

Algo que tiene que tener en cuenta es que siempre que haga dibujos, si la figura es cerrada, deberá emplear el método closePath() para unificar con una línea recta el punto final con el inicial.

Los métodos de dibujo de texto, tienen un tratamiento diferente. Se materializan en el acto, ya sea con contorno y/o relleno. Por lo tanto, no cuentan como caminos de trazos. Se renderizan instantáneamente. Por ello los nombres de sus métodos son fillText y strokeText de manera directa.

1. Construya una función que sea capaz de dibujar una cruz diagonal (cruzando los vértices) de toda el área de dibujo.
2. Construya una función que le permita dibujar un triángulo dado sus tres vértices en el centro del área de dibujo.
3. Construya una función que le permita dibujar cualquier polígono cerrado ubicado en el centro del área de dibujo, dada una lista ordenada de puntos (x,y).
4. Configurar el área de dibujo para tener forma de cuadrado de (500,500)px. Empleando las primitivas de dibujo de arcos de elipse, dibujar círculos concéntricos hasta el centro del área, espaciados en 25px c/u.
5. Reutilizando el código del punto anterior, y empleando las primitivas de dibujo de texto. Construir un tablero de dardos, en donde se reparta el área del cuadrado para que entren hasta 5 círculos concéntricos. Entre el espacio de cada círculo, se deberá distribuir el valor de puntaje según el array: [1000, 750, 500, 100, 50] donde el centro representa el mayor valor.
6. **Integración:** Modificar el WebComponent de modo que el modelo le permita agregar dinámicamente (círculos y polígonos) ubicables en cualquier punto (x,y) del área de dibujo, como así también, limpiar toda el área de dibujo. El componente tendrá que tener un botón de carga de figuras, que al realizar click, lance un *prompt* esperando el objeto JSON para representarlo inmediatamente en el canvas. También deberá tener un botón de limpieza.
7. Dado un objeto de datos: [0.30, 0.50, 0.10, 0.10] que representa información porcentual, elaborar una función que procese el array y dibuje la representación visual como un gráfico de torta mostrando el valor del porcentaje como texto.
8. Construya un objeto de datos acorde más flexible, que le permita al dibujador de gráficos de torta mostrar una etiqueta de texto personalizada para cada porción junto a su porcentaje.

Capítulo 2: Bordes, estilos y tipos de línea

Hasta este punto, se ha familiarizado con los conceptos de dibujo, trazados básicos y composición de figuras. Ha utilizado dos métodos de renderización (stroke/fill) adecuadamente de forma simplificada. La Canvas API ofrece un rico conjunto de renderizados de líneas que le permiten complejizar más sus dibujos.

Propiedad / Método	Tipo / Valores comunes	Descripción
lineWidth	Propiedad (number)	Define el grosor de la línea en píxeles. El valor por defecto es 1.0.
lineCap	Propiedad ('butt' 'round' 'square')	Define el remate o acabado visual de los extremos de una línea abierta: • 'butt' (por defecto): Extremo plano recto al borde exacto. • 'round': Extremo redondeado. • 'square': Extremo cuadrado que extiende la línea medio ancho más.
lineJoin	Propiedad ('miter' 'round' 'bevel')	Define el aspecto visual de la esquina donde se unen dos segmentos de línea: • 'miter' (por defecto): Esquina en punta o en ángulo agudo. • 'round': Esquina redondeada. • 'bevel': Esquina biselada (plana/triangular).

miterLimit	Propiedad (number)	Establece la longitud máxima del pico de una unión 'miter' antes de convertirse automáticamente en bisel ('bevel'). Evita picos desproporcionadamente largos en ángulos muy agudos. El valor por defecto es 10.0.
setLineDash(segments)	Método (Array<number>)	Establece el patrón de punteado o trazo discontinuo. Recibe un array de números que alternan distancias de línea pintada y espacio en blanco (ej: [5, 15]). Pasar un array vacío [] vuelve a la línea sólida.
getLineDash()	Método (Devuelve Array<number>)	Devuelve el array de números que representa el patrón de punteado actualmente activo.
lineDashOffset	Propiedad (number)	Define el desplazamiento (desfase) del patrón de punteado. Se utiliza frecuentemente para animar líneas punteadas (efecto de "hormigas marchando").

9. Construir un muestrario de líneas verticales de grosor creciente, variando desde 1px hasta 25px con un espaciado de 16px entre cada una. Cada línea debe tener el mismo alto del área de dibujo.
10. Dibujar un conjunto de tres líneas quebradas independientes (cada una formada por 3 puntos) aplicando en cada una un estilo diferente de terminación de extremos (butt, round, square) y de unión de esquinas (bevel, round, miter).
11. Líneas punteadas (setLineDash): Construir una función que dibuje una grilla de líneas diagonales separadas cada 15px, alternando patrones de trazos continuos y punteados.
12. **Integración:** Modificar el modelo del WebComponent integrado (Ej:6), de modo que las figuras gestionadas admitan propiedades de tipo de línea y grosor. Para esto, el componente deberá tener dos inputs para seleccionar el tipo de línea y el valor de grosor. Al hacer click en el botón de carga de figuras, se seguirá aceptando el mismo JSON sin cambios, solo anexará el valor de los inputs a la estructura de datos.

Capítulo 3: Colores y gradientes

El color en Canvas 2D es el elemento fundamental para transformar estructuras geométricas simples en composiciones visuales complejas y expresivas. A través del modelo de estado de la API, el lienzo gestiona el color de manera global mediante las propiedades *fillStyle* y *strokeStyle*, las cuales aceptan cualquier formato válido del estándar CSS (nombres clave, Hex, RGB, HSL y sus variantes con canal alfa para transparencias). Más allá de las tonalidades planas, la API ofrece la capacidad de generar dinámicamente gradientes lineales (*createLinearGradient*), radiales (*createRadialGradient*) y cónicos (*createConicGradient*), así como la aplicación de patrones repetitivos (*createPattern*) a partir de imágenes o fragmentos del propio lienzo, aunque esto será desarrollado más adelante. Comprender la creación, distribución de paradas de color (*color stops*) y asignación de estas estructuras resulta clave para dotar a las primitivas de profundidad, volumen y un acabado gráfico avanzado.

Categoría	Método / Propiedad	Descripción
Asignación de Relleno	fillStyle = string Gradient Pattern	Define el color, gradiente o patrón para el interior de las figuras. Acepta cualquier formato CSS válido (ej: 'red', '#ff0000', 'rgb(255,0,0)', 'rgba(0,0,0,0.5)'). String corresponde a relleno sólido.

Asignación de Borde	<code>strokeStyle = string gradient pattern</code>	Define el color, gradiente o patrón para el contorno de las figuras. Acepta cualquier formato CSS válido.
Opacidad Global	<code>globalAlpha = 0.0 .. 1.0</code>	Define el nivel de transparencia de todo lo que se dibuje a continuación sobre el lienzo. El valor por defecto es 1.0 (completamente opaco).
Gradiente lineal	<code>createLinearGradient(x0, y0, x1, y1)</code>	Crea la estructura de un gradiente lineal a lo largo de un eje definido por dos puntos (x0,y0) a (x1,y1).
Gradiente radial	<code>createRadialGradient(x0, y0, r0, x1, y1, r1)</code>	Crea la estructura de un gradiente radial entre dos círculos concéntricos o desplazados.
Gradiente cónico	<code>createConicGradient(startAngle, cx, cy)</code>	Crea la estructura de un gradiente cónico (angular) alrededor de un punto central.
Gradiente por tramos	<code>gradient.addColorStop(offset, color)</code>	Añade una parada de color al gradiente. offset va de 0.0 (inicio) a 1.0 (fin).
Patrones	<code>createPattern(image, repetition)</code>	Crea un patrón repetitivo a partir de un objeto de imagen, otro canvas o un video. Modos: 'repeat', 'repeat-x', 'repeat-y' o 'no-repeat'.

13. Construir un tablero simil de ajedrez (8x8) casillas cuadradas de 32px de ancho. Colocar en la interfaz (por fuera del canvas) un slider que para cada componente de color RGB. Todos las casillas del tablero que contendrían la coloración negra, deberán aplicar el color según la configuración RGB de los sliders. *Utilice el slider (WebComponent) desarrollado en unidades anteriores.*
14. Construir un rectángulo de (200px, 400px) coloreado mediante un gradiente lineal horizontal entre dos colores diferentes empleando la notación HEX. Ambos colores deben ser provistos por dos inputs de color por fuera del canvas.
15. Utilizando la función constructora de polígonos desarrollada en el capítulo uno, construya otra función que aplique un gradiente aplicado entre dos colores en notación HEX, con un ángulo de 45° de inclinación.
16. Dibujar un círculo relleno mediante un gradiente radial donde el centro y el borde exterior compartan el mismo punto focal.
17. Dibujar un cilindro y rellenar con gradientes cada región de modo que simule el efecto de volumen 3D.
18. Construir un cuadrado coloreado con un gradiente cónico con origen en el centro de la figura, cubriendo los 360° con una lista de tres colores.
19. **Integración:** Tomando la integración del punto 12, agregar un botón selector (select) desplegable que permita seleccionar la figura que desee cargar (Círculo, Polígono) e incorpore un nuevo objeto llamado “*CakeGraph*” (Gráfico de torta circular) de modo que procese un JSON que contenga en su interior la descripción de datos que corresponda para representar un gráfico de tortas. Cada región (tajada) se rellena con un color aleatorio de forma automática. Es decir, no debe suministrar colores.

Capítulo 3: Texturas

La manipulación de texturas en Canvas 2D permite trascender los colores planos y los gradientes geométricos para poblar la superficie del lienzo con contenido visual complejo, fotorrealista o repetitivo. A través de la integración de fuentes de datos de mapa de bits —como objetos de imagen (HTMLImageElement), elementos de video (HTMLVideoElement) u otros lienzos gráficos independientes—, la API posibilita cargar, renderizar y proyectar imágenes sobre cualquier primitiva mediante el método `drawImage()`. Asimismo, la creación de patrones repetitivos a través de `createPattern()` habilita el relleno de formas con texturas continuas (mosaicos). Esta capacidad resulta fundamental para la construcción de fondos en videojuegos, la composición de materiales visuales, la renderización de *sprites* y la superposición de elementos gráficos dinámicos en tiempo real.

Categoría	Método / Propiedad	Descripción
Carga	<code>new Image()</code>	Crea un elemento de imagen HTML en memoria. Se utiliza para cargar una textura mediante su propiedad <code>src</code> antes de dibujarla.
	<code>image.onload</code>	Evento imprescindible que se ejecuta cuando la textura termina de cargarse en memoria. Todo intento de dibujar la imagen debe ocurrir dentro o después de este evento.
Renderizado simple	<code>drawImage(image, x, y)</code>	Dibuja la textura completa en su tamaño original (en píxeles) posicionando su esquina superior izquierda en (x, y).
Renderizado con escalado	<code>drawImage(image, x, y, ancho, alto)</code>	Dibuja la textura en la posición (x, y) redimensionándola al ancho y alto especificados (escala/deforma la imagen).
Renderizado con recorte	<code>drawImage(image, sx, sy, sAncho, sAlto, dx, dy, dAncho, dAlto)</code>	Recorta un área de la textura fuente en (sx, sy) de tamaño sAncho × sAlto y la dibuja en el lienzo en (dx, dy) con tamaño dAncho × dAlto. Ideal para spritesheets.
Patrón Repetitivo	<code>createPattern(image, repeticion)</code>	Crea una textura de mosaico a partir de una imagen, video o canvas. Los valores de repeticion son: 'repeat', 'repeat-x', 'repeat-y', o 'no-repeat'.
Calidad de Suavizado	<code>imageSmoothingEnabled = true false</code>	Propiedad que activa (true) o desactiva (false) el suavizado bilineal al escalar imágenes. Desactivarlo es clave para gráficos estilo Pixel Art.
Calidad de Suavizado	<code>imageSmoothingQuality = 'low' 'medium' 'high'</code>	Establece el nivel de calidad de los algoritmos de interpolación aplicados al escalar las texturas.
Filtros Gráficos	<code>filter = "effect(%) effect(%)..."</code>	Aplica filtros de procesamiento visual en tiempo real a las texturas (similar a CSS). Acepta: <code>blur()</code> , <code>brightness()</code> , <code>contrast()</code> , <code>drop-shadow()</code> , <code>grayscale()</code> , <code>hue-rotate()</code> , <code>invert()</code> , <code>opacity()</code> , <code>saturate()</code> y <code>sepia()</code> . Usa 'none' para desactivar.

20. Cargar una imagen mediante la API de `Image()` y dibujarla completa dentro del canvas en una posición específica.
21. Cargar una imagen y dibujarla en modo escalado de modo que ocupe todo el ancho y alto del área de dibujo.
22. Utilizar la variante de 9 parámetros de `drawImage(image, sx, sy, sWidth, sHeight, dx, dy, dWidth, dHeight)` para tomar solo una región cuadrada del centro de la imagen y dibujarla ampliada en el canvas.

23. Cargar una imagen de tamaño pequeño y utilizar `createPattern(image, 'repeat')` como `fillStyle` para rellenar el interior de un rectángulo o polígono.
24. Dibujar un círculo o polígono en el canvas, definirlo como máscara mediante la función `clip()` y luego renderizar la imagen para que solo sea visible dentro de la figura recortada.
25. Cargar una imagen y dibujarla en el canvas dentro de un bloque `save()` y `restore()` aplicando una combinación de al menos tres filtros nativos simultáneos (por ejemplo: brillo al 120%, contraste al 150% y un desenfoque de 2px), garantizando que los filtros aplicados no afecten a las siguientes figuras que se dibujen en el canvas.
26. **Integración:** Extienda el WebComponent desarrollado de modo que el selector ofrezca la posibilidad de cargar imágenes. El modo de carga se realizaría mediante URL válida.

Capítulo 4: Transformaciones geométricas

Las transformaciones geométricas en Canvas 2D permiten alterar la posición, orientación y escala del sistema de coordenadas sobre el cual se dibuja, en lugar de modificar las coordenadas de cada figura de forma individual. A través de operaciones fundamentales como la traslación (`translate`), la rotación (`rotate`) y el escalado (`scale`), es posible desplazar el origen del lienzo, girar elementos en torno a un punto de pivote o modificar sus proporciones de manera dinámica. Para gestionar estas modificaciones sin afectar al resto de la composición, la API utiliza una pila de estados mediante los métodos `save()` y `restore()`, los cuales permiten aislar las transformaciones de un objeto específico y regresar al estado original. Dominar este enfoque matricial es indispensable para estructurar animaciones complejas, componer sistemas jerárquicos o renderizar sprites y objetos con orientaciones dinámicas en pantalla.

Durante el capítulo 1, se hizo una observación con respecto a la limitación del uso de coordenadas absolutas. Piense en lo siguiente. Imagine que dibuja una casa simple con coordenadas absolutas. De alguna manera, usted define con exactitud la ubicación de la puerta, las ventanas, el techo y las paredes. Ahora imagine que quiere construir otra casa en otro lugar del área de dibujo. Debería reconsiderar el recálculo de todas las coordenadas para la nueva puerta, nuevo techo y demás. En cambio, si usted dibujara largos y anchos respecto a un punto específico. Todas las medidas quedarían en función del punto base. Por lo tanto, no importa dónde quiera colocar la figura, siempre quedará abstraído de la posición. Esta relativización se consigue mediante la operación de traslación.

27. Dibujar un rectángulo en el centro del área de dibujo y aplicar una rotación de 45 grados (utilizando `Math.PI / 4`).
28. Construir una función que combine en orden estricto `translate(x, y)` y `rotate(ángulo)` para lograr que un polígono rote sobre su propio centro geométrico.
29. Dibujar una elipse base y aplicar `scale(sx, sy)` utilizando factores de escala no proporcionales (por ejemplo 2.0 en ancho y 0.5 en alto), observando el efecto en la figura y en el grosor del borde (`lineWidth`).
30. Deformación lateral horizontal (Skew X): Utilizar la matriz de transformación `ctx.transform(1, 0, tanX, 1, 0, 0)` para aplicar una inclinación oblicua sobre el eje X a un cuadrado, convirtiéndolo en un paralelogramo.
31. Deformación lateral vertical (Skew Y): Utilizar la matriz de transformación `ctx.transform(1, tanY, 0, 1, 0, 0)` para aplicar una inclinación oblicua sobre el eje Y a un rectángulo, simulando una proyección de perspectiva lateral.
32. **Integración:** Tomando la integración anterior, implemente un manejador de eventos que le permita obtener la posición del mouse sobre el área de dibujo y que al hacer click sobre el área, el controlador solicite la carga de la

figura que indique el selector en ese momento para ser dibujada específicamente en la coordenada del click. Adecúe el WebComponent para que el renderizado utilice una operación de traslación (`save()`, `translate(x, y)` y `restore()`) para realizar el dibujo de cualquier figura seleccionable.

Capítulo 5: Animaciones e introducción al modelado físico

La animación y el modelado físico representan el paso decisivo de la gráfica estática al comportamiento dinámico e interactivo en Canvas 2D. En lugar de ejecutar trazados fijos, la animación se fundamenta en un ciclo continuo de renderizado (*game loop*) impulsado por el objeto temporizador invocable `requestAnimationFrame()`, donde el estado del lienzo se borra, se recalculan las variables de posición y se redibuja cada fotograma a la tasa de refresco del navegador. Al integrar conceptos básicos de física —como velocidad, aceleración, fricción, gravedad y detección de colisiones—, los objetos gráficos dejan de moverse en trayectorias rígidas y pasan a reaccionar bajo fuerzas fotorrealistas. Dominar la sincronización de cuadros, el cálculo de deltas de tiempo y la simulación de leyes físicas es la base esencial para la creación de videojuegos 2D, sistemas de partículas y simuladores mecánicos interactivos.

El concepto clave de la animación siempre consiste en dibujar un fotograma completo, limpiarlo y volverlo a redibujar a una tasa de velocidad razonable. Generalmente el estándar se considera en 60 fotogramas (redibujados) por segundo. Este parámetro responde a la frecuencia en la que el ojo humano captura movimiento fluido. Por debajo de este valor podría percibir una animación como “trabada” visualmente. En juegos, resulta la tasa óptima. Sin embargo todo depende del tipo de aplicación específica y de las capacidades del hardware para poder sostener ese nivel de fluidez.

En aplicaciones gráficas de programas de escritorio y sin presencia de animaciones, se suele renderizar bajo demanda de interacciones en partes específicas de la pantalla para ahorrar recursos y disminuir el ancho de banda de datos con la CPU en caso de que la renderización se haga vía software. En caso de que se necesita cierta fluidez constante, las librerías gráficas 2D suelen acelerar por hardware redirigiendo los comandos de dibujo de bajo nivel a la GPU. Un panorama distinto se da cuando se emplean contextos 3D donde la aceleración por hardware es indispensable en el 99% de los casos. En el contexto web, toda esta complejidad está abstraída dado que la API es de muy alto nivel.

Entre cada redibujado de fotograma, la aplicación generalmente efectúa cálculos sobre el conjunto de elementos gráficos desplegados en pantalla para determinar sus nuevas posiciones y/o cambios gráficos en función de un período de tiempo artificial (llamado delta), aparezca una sutil diferencia que a la velocidad de 60 fotogramas por segundo, denote una ilusión de movimiento continuo.

La animación en sí, es sólo eso. Generar la ilusión de movimiento. Ahora la forma de gestionar ese cálculo de posiciones y geometrías de los objetos en pantalla puede hacerse de muchas maneras. De manera completamente arbitraria y manual, computable por generador de coordenadas, por modelado físico (donde se introducen conceptos de velocidad, aceleración) y que puede ser tan realista como se desee ó bien una combinación híbrida de todas.

En nuestra introducción, sólo abordaremos el concepto de generador y física elemental. En concreto emplearemos los principios básicos del modelo de cuerpo rígido con cinemática tipo MRUV (Movimiento rectilíneo uniformemente variado) o denominado de aceleración constante.

Asumamos que tenemos un cuadrado de 50px de lado sobre un área de dibujo de 640px x 480px.



Las propiedades de este cuadrado (rectángulo) sin presencia de animación podrían resumirse en: posición en eje X, Y y ancho/alto. Asumamos que la posición inicial es (x=100, y=100).

La animación debería actualizar (x,y) en una pequeña cantidad de movimiento. A esta pequeña cantidad la podemos llamar *diferencial de desplazamiento* (dx, dy).

Si necesitamos que el objeto se mueva instantáneamente sobre el eje X y a velocidad fija, alcanzaría con considerar por ejemplo que (dx = 2, dy = 0).

Esto significa que luego del dibujado del fotograma, las posiciones se deberían recalcular como:

```
cuadrado.x += dx  
cuadrado.y += dy
```

De esta manera, para el próximo fotograma, el cuadrado se habrá desplazado 2px en el eje horizontal y 0 en el vertical. Es el modelo más simple, y se llama animación lineal a velocidad constante.

Si quisiéramos configurar algo sutilmente más realista, podría ser: "Que el objeto tenga una rampa de velocidad, arrancando desde 0 hasta un límite". El cálculo en este caso debería replantearse como:

```
cuadrado.x += dx(t)  
cuadrado.y += dy(t)
```

El asunto es que ese desplazamiento es función del tiempo, por ende del fotograma. Al principio valdrá 0, luego irá incrementándose 1, 2, 4, 6.. etc. ¿Cómo saber cuánto vale? Hay numerosas formas de implementar esto. Una que resulta muy simple es aplicar la función de cinemática de MRUV.

$$a(t) = dv/dt \rightarrow a = dv/dt \rightarrow dv = a \cdot dt \quad (1)$$

Si decimos que la aceleración es constante, entonces a deja de depender de t, y como dv (diferencial de velocidad) es el diferencial de desplazamiento (dx) respecto al diferencial de tiempo dt entonces:

$$\begin{aligned} dv &= a \cdot dt \\ d(dx/dt) &= a \cdot dt \\ d^2x/dt^2 &= a \cdot t \\ d^2x &= a \cdot dt^2 \\ dx &= (a \cdot t + v0) dt \\ dx &= v0 + v(t) \cdot dt \quad (2) \end{aligned}$$

De las ecuaciones (1) y (2) se deduce e infiere conclusiones muy importantes. La variación de velocidad entre cada cada tiempo de generación de fotograma (dt) varía proporcionalmente con la aceleración constante. Y por otro lado, el desplazamiento (dx) depende de la velocidad y del tiempo del fotograma (dt).

En consecuencia:

- No se puede obtener el desplazamiento (dx) para un un tiempo (dt), si no se conoce la velocidad que tenía en el fotograma anterior.
- No se puede obtener la velocidad si no se conoce la aceleración constante de ese objeto.
- Si la aceleración es constante entonces es fija para toda la vida del objeto.

Por lo tanto, el objeto cuadrado debería reformularse para tener algunas propiedades adicionales, quedando constituido por: { x, y, v, a }. De esta forma ahora el cálculo se redefine como:

```
cuadrado.a = 2
cuadrado.v += cuadrado.a · dt
cuadrado.x += cuadrado.v · dt
```

Así se consigue y demuestra la cinemática más elemental de objetos animados con aceleración constante.

33. Adaptar las propiedades de las figuras para que puedan adquirir capacidad de movimiento. Construir un loop de renderizado invocando el objeto temporizador `requestAnimationFrame()` para una velocidad de 60fps. Asumir que todos los objetos tienen aceleración y velocidad inicial 0 para todas las direcciones (x e y).
34. Crear un rectángulo y realizar la animación mediante MRUV.
35. Crear un cuadrado y realizar la animación mediante MRUV considerando ambos ejes (x e y)
36. Construir un círculo de 16px a 100px del centro y realizar una animación en donde el objeto se mueva orbitando el centro.
37. Construir en base a una imagen tipo “spritesheet” una animación en donde se recorre cada subfotograma de la tira animada de modo que simule un movimiento.